

Architettura degli Elaboratori

L'architettura della CPU – Introduzione alla pipeline



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo

Masi, Claudio di Ciccio

[S&PdC] 4.5





Esercizio per casa: vj

Si vuole aggiungere alla CPU l'istruzione vectorised jump (**vj**), di tipo I e sintassi assembly

vj *indice*, *x0*, *vettore*

che salta all'indirizzo contenuto nell'elemento indice-esimo del *vettore* di word. Il secondo registro (*rs2*) non è utilizzato e per questo viene settato a zero.

L'assemblatore inserirà nella parte immediata, i 12 bit di scostamento relativi al program counter per raggiungere l'indirizzo di memoria dell'etichetta *vettore*. In altre parole, l'indirizzo di memoria dell'etichetta è uguale $PC + [20-31]$ (i 12 bit dell'istruzione).

Esempio: se in memoria si è definito staticamente il

vettore: **.word** 0x10008000, 0x10008200, 0x10001088, 0x10008444

e al registro **t0** è assegnato il valore **3** allora

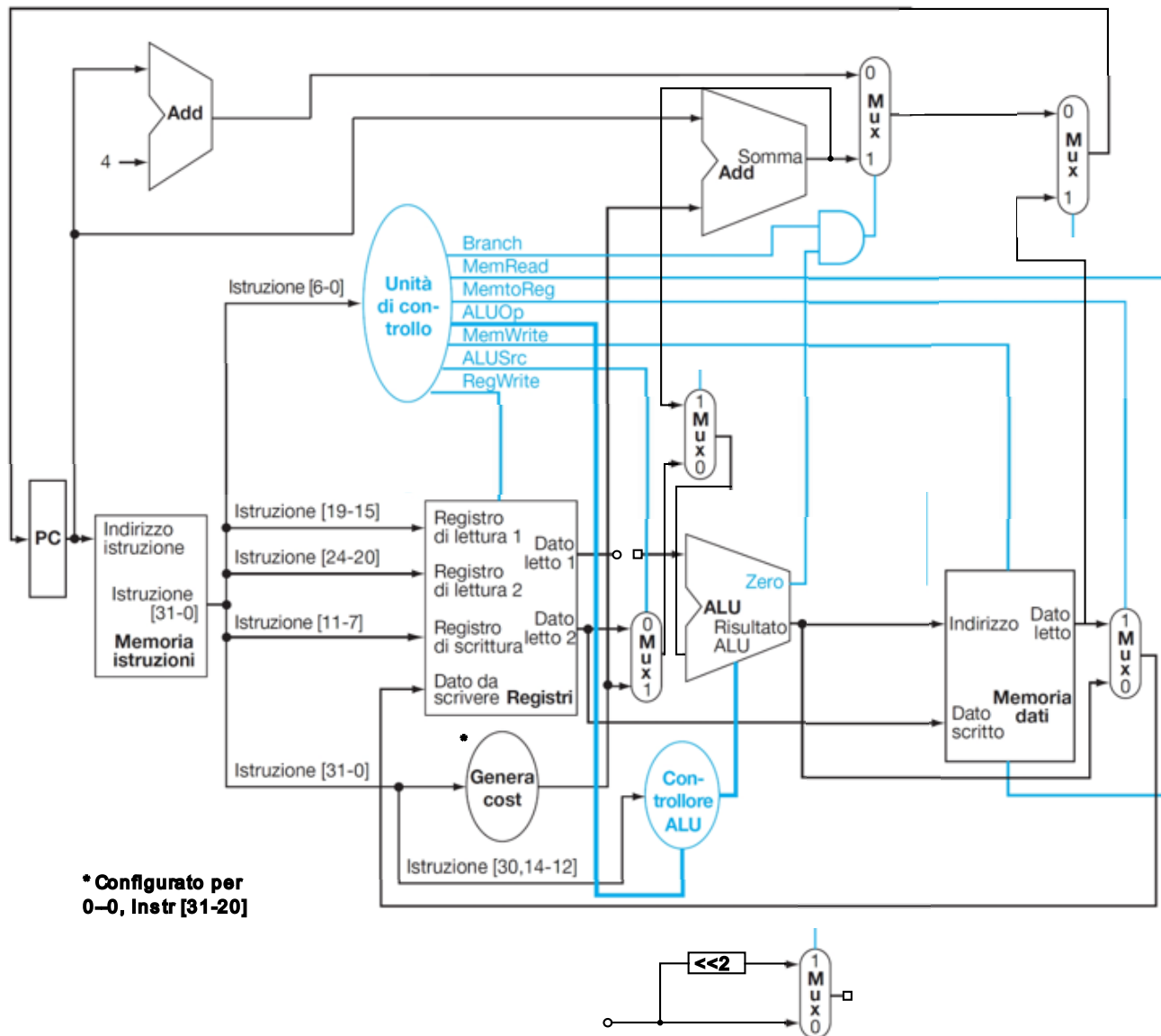
vj *t0*, *vettore*

salterà all'indirizzo **0x10008444** (che è l'elemento con indice 3 del vettore).

- si disegnino le modifiche necessarie a realizzare la funzione, aggiungendo tutti gli eventuali MUX, segnali di controllo, bus, ALU e sommatori (ecc) che ritenete necessari.
- indicate i valori di tutti i segnali di controllo, in modo da eseguire l'istruzione **vj**.
- tenendo conto che il tempo di accesso ai registri (sia in lettura che scrittura) è di **25ns**, l'accesso alla memoria impiega **75ns**, la ALU e i sommatori impiegano **100ns** e ignorando gli altri ritardi, calcolate il tempo di esecuzione minimo della istruzione **vj** e indicate se è necessario aumentare il periodo di clock della CPU per poter svolgere questa nuova istruzione.

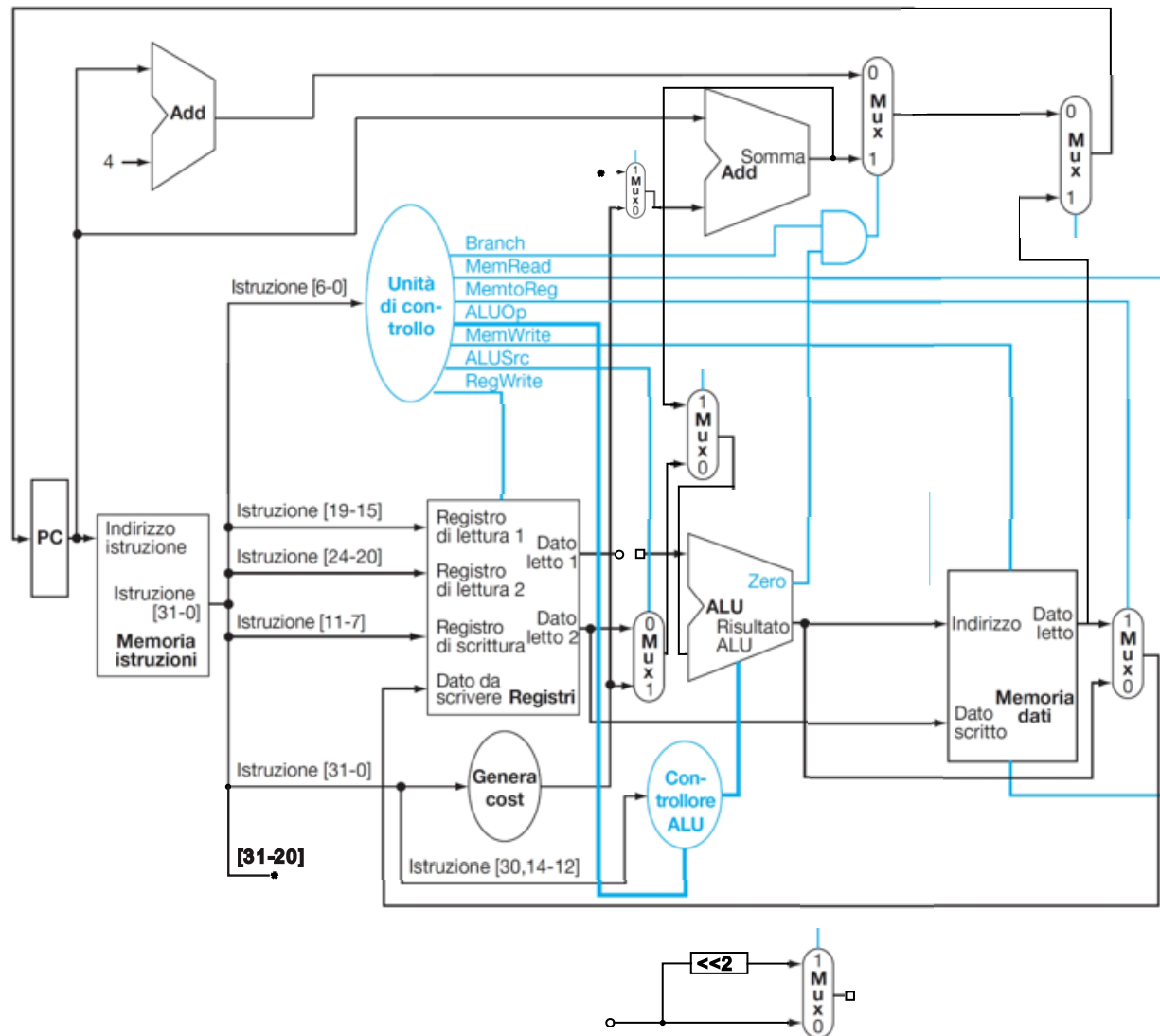
vj indice, *vettore*

salta all'indirizzo contenuto nell'elemento
indice-esimo del *vettore* di **word**



vj indice, *vettore*

soluzione senza usare Genera cost





Esercizio per casa: Diff and store

Considerare l'architettura RISC-V a ciclo singolo. Si vuole aggiungere l'istruzione di tipo `diff_and_store rs1, rs2, addr`

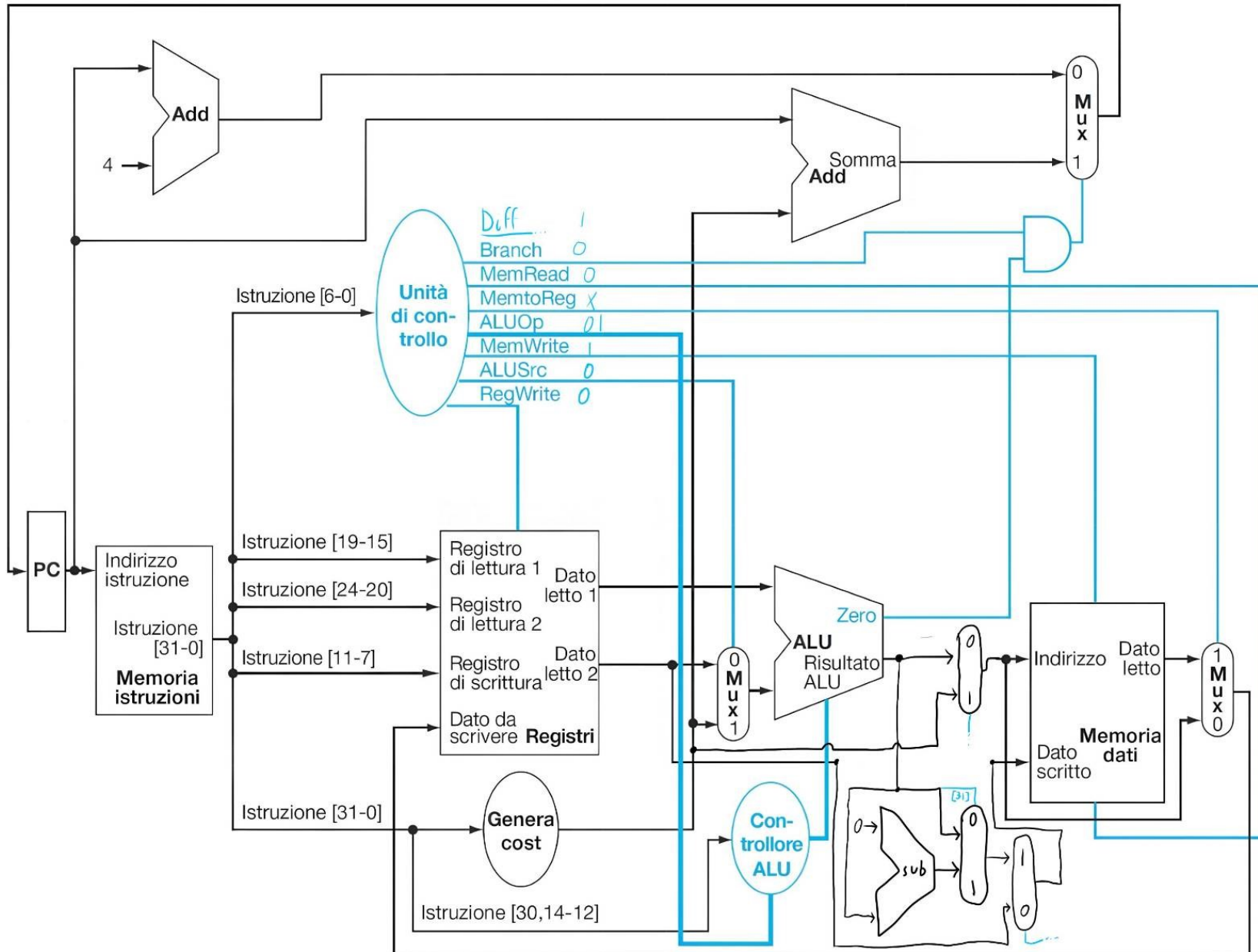
(calcola la differenza in valore assoluto dei registri `rs1` ed `rs2` e salva il risultato in memoria, all'indirizzo `addr`)

che equivale al seguente frammento di codice:

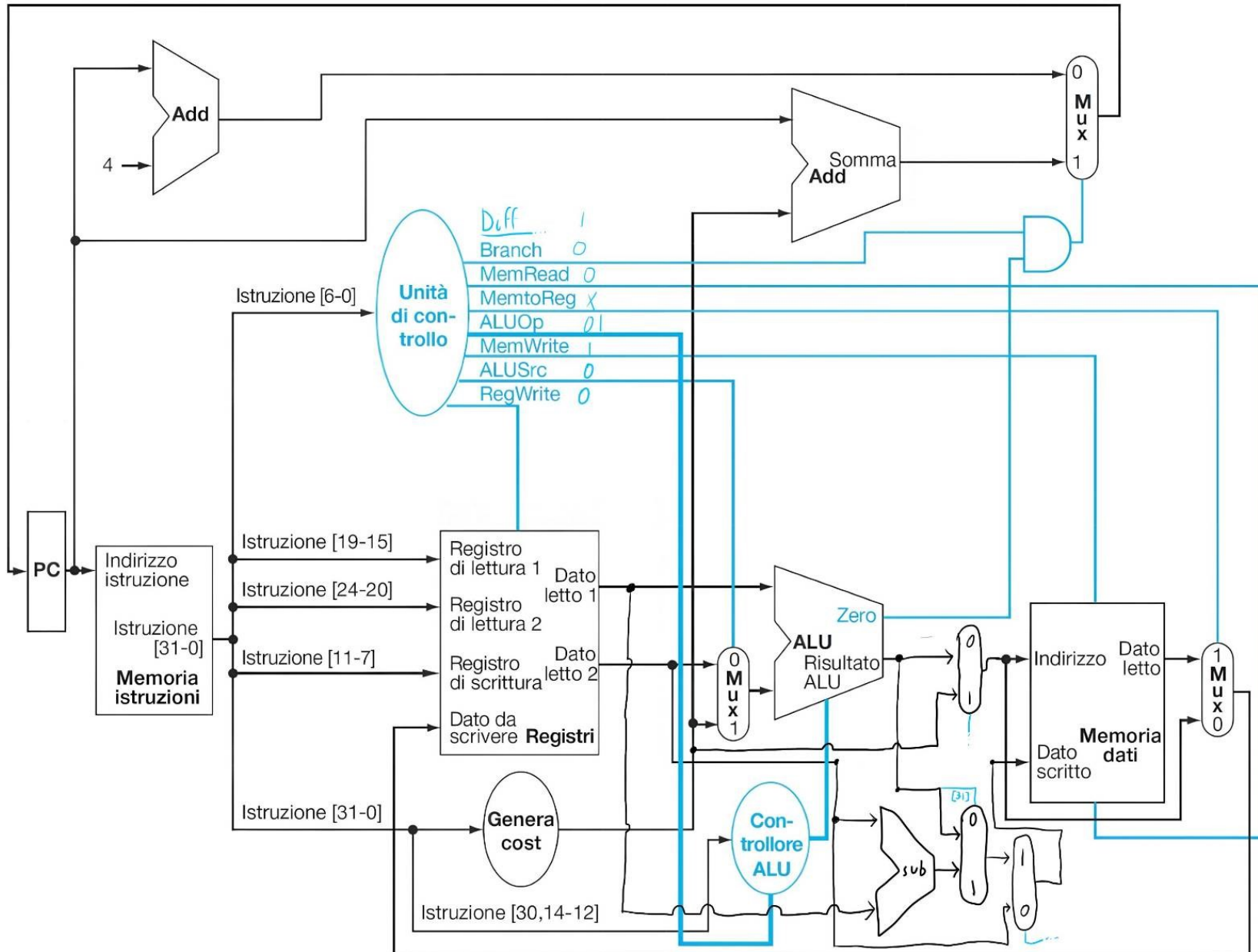
```
sub rs1, rs1, rs2      # (a)
bge rs1, x0, save      # (b)
sub rs1, x0, rs1       # (c)
save:
    sw rs1, addr       # (d)
```

ossia (a) esegue la sottrazione tra `rs1` ed `rs2`, (b,c) se il risultato è negativo ne cambia il segno, infine (d) salva il valore risultante in memoria all'indirizzo indicato da `addr`. In particolare, `addr` è la parte immediata dell'istruzione e viene usato come indirizzo assoluto di destinazione. L'assemblatore tratterà la parte immediata *addr* come se l'istruzione fosse una `sw`.

diffandstore rs1,rs2,addr



diffandstore rs1,rs2,addr (soluzione più efficiente)



Argomenti

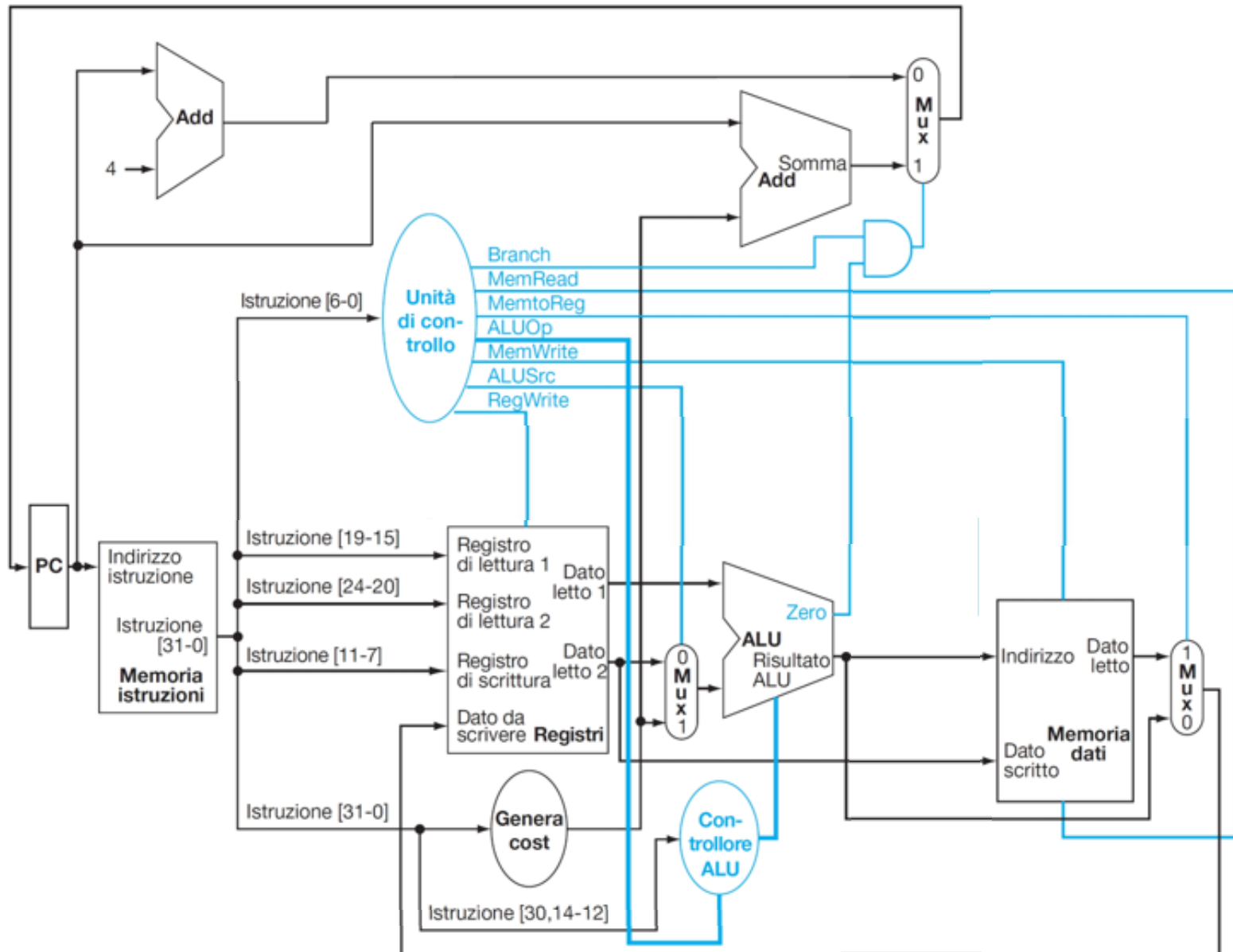
Argomenti della lezione

- Le 5 fasi dell'istruzione
- Introduzione della pipeline
 - Solo una fase è attiva in ogni istante
 - Pipeline per processare più istruzioni contemporaneamente
 - Hazard sui dati e sul controllo

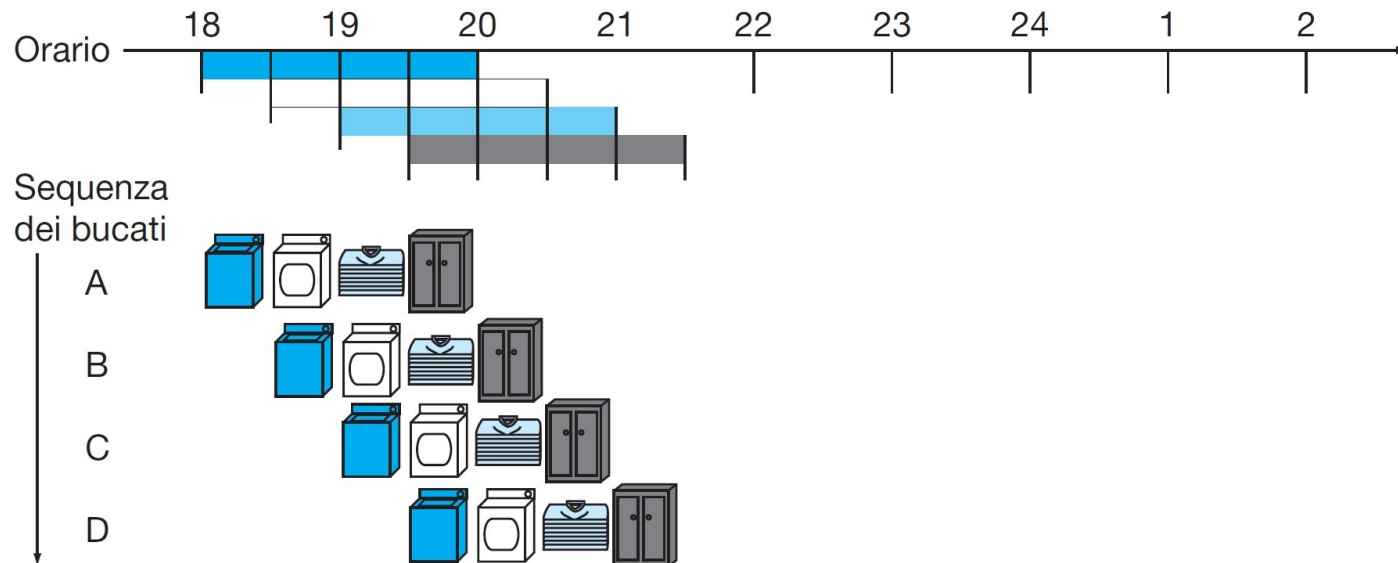
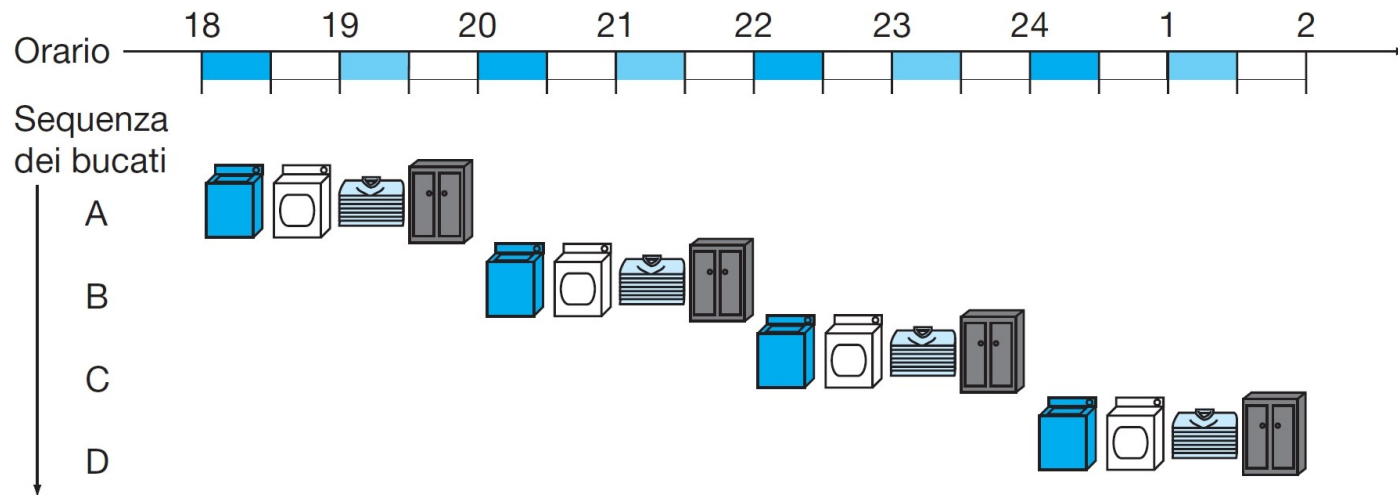
Fasi dell'istruzione:

- **Fetch:** carica l'istruzione dalla memoria
- **Instruction Decode:** la CU decodifica l'istruzione e vengono letti gli argomenti dai registri
- **Execution:** l'ALU fa il calcolo necessario (tipo R o accesso alla memoria o branch)
- **Memory access:** viene letta/scritta la memoria (lw e sw)
- **Write Back:** il risultato dell'ALU o quello letto dalla memoria viene messo nel registro dest.

Riepilogo – CPU RISC-V a un ciclo di clock

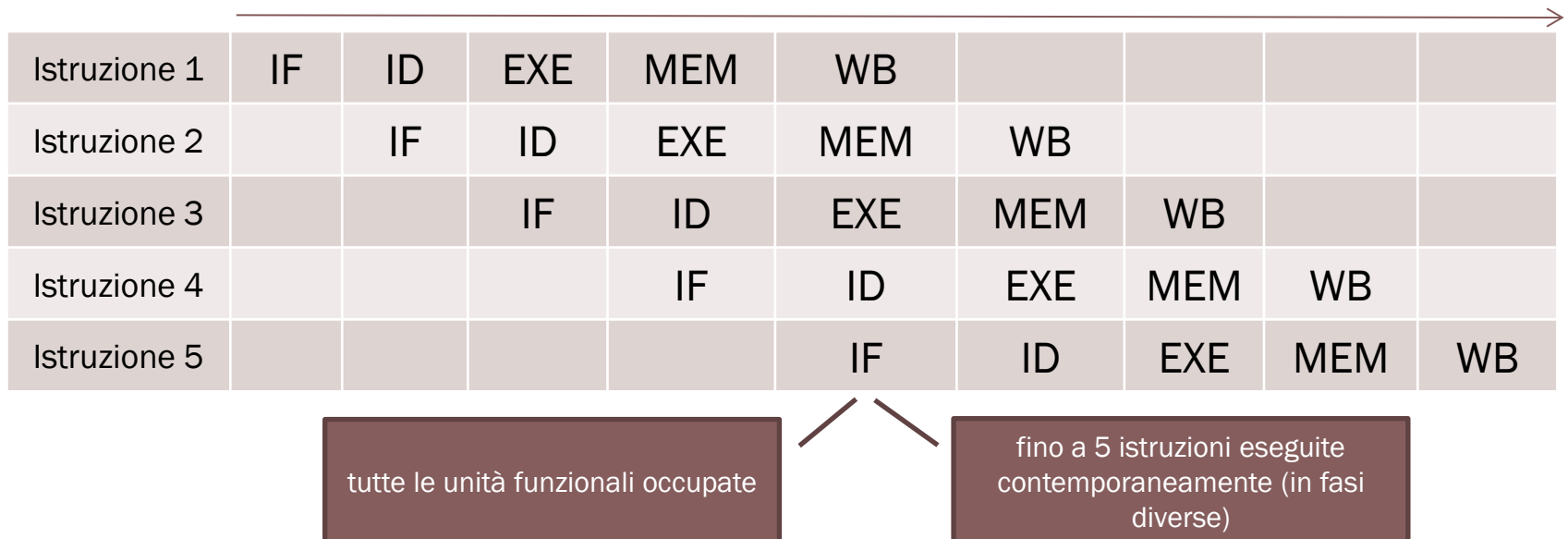


Esempio di pipeline



La CPU è per l'80% inutilizzata!

- In ogni momento solo un'unità funzionale è attiva
 - Instruction Fetch (IF): **Memoria Istruzioni (e aggiornamento PC)**
 - Instruction Decode (ID): **Blocco registri (e CU)**
 - Execute (EXE): **ALU**
 - Memory access (MEM): **Memoria dati**
 - Write Back (WB): **Banco registri**
- **Idea:** trasformare la CPU in una catena di montaggio
 - ogni unità funzionale elabora la fase che gli corrisponde e passa l'istruzione alla fase successiva



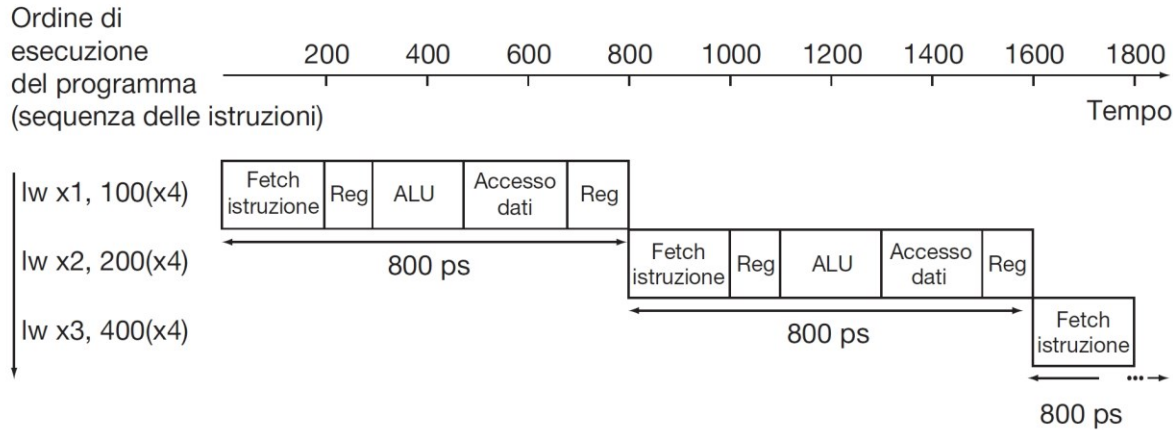
Incremento della velocità

- Le 5 fasi devono sovrapporsi
 - individuare il periodo di clock uniforme per svolgere ognuna (durata della fase più lenta)

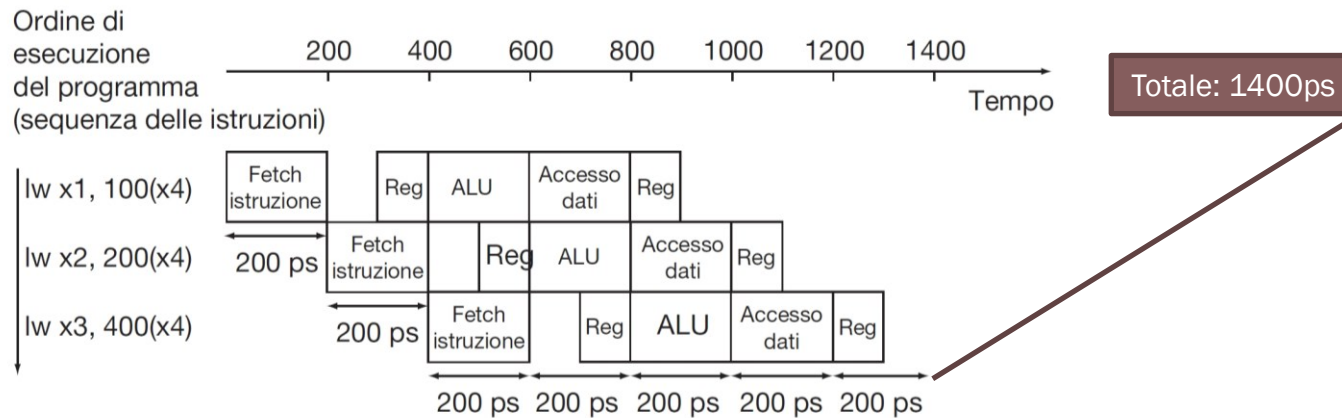
Classe dell'istruzione	Lettura dell'istruzione	Lettura dei registri	Operazione con la ALU	Accesso ai dati in memoria	Scrittura del register file	Tempo totale
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
Formato R (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Salto condizionale (beq)	200 ps	100 ps	200 ps			500 ps

- Possiamo ridurre il periodo di clock da **800 ps** (durata massima di una istruzione) a **200 ps** (durata massima di una fase)
 - periodo di clock ridotto ad un quarto → velocità quadruplicata
 - (in pratica, si cerca sempre di progettare la CPU in modo che le fasi abbiano tempi simili)
 - se tutte le fasi avessero tempi uguali si potrebbe aumentare la velocità di 5 volte
- Ad ogni colpo di clock una istruzione viene completata dalla pipeline
 - Possiamo arrivare a **quintuplicare (idealmente) il throughput (#istruzioni / fase)**
 - Il tempo per eseguire una singola istruzione è lo stesso
(o addirittura maggiore se tempi non simili)

Esempio di esecuzione

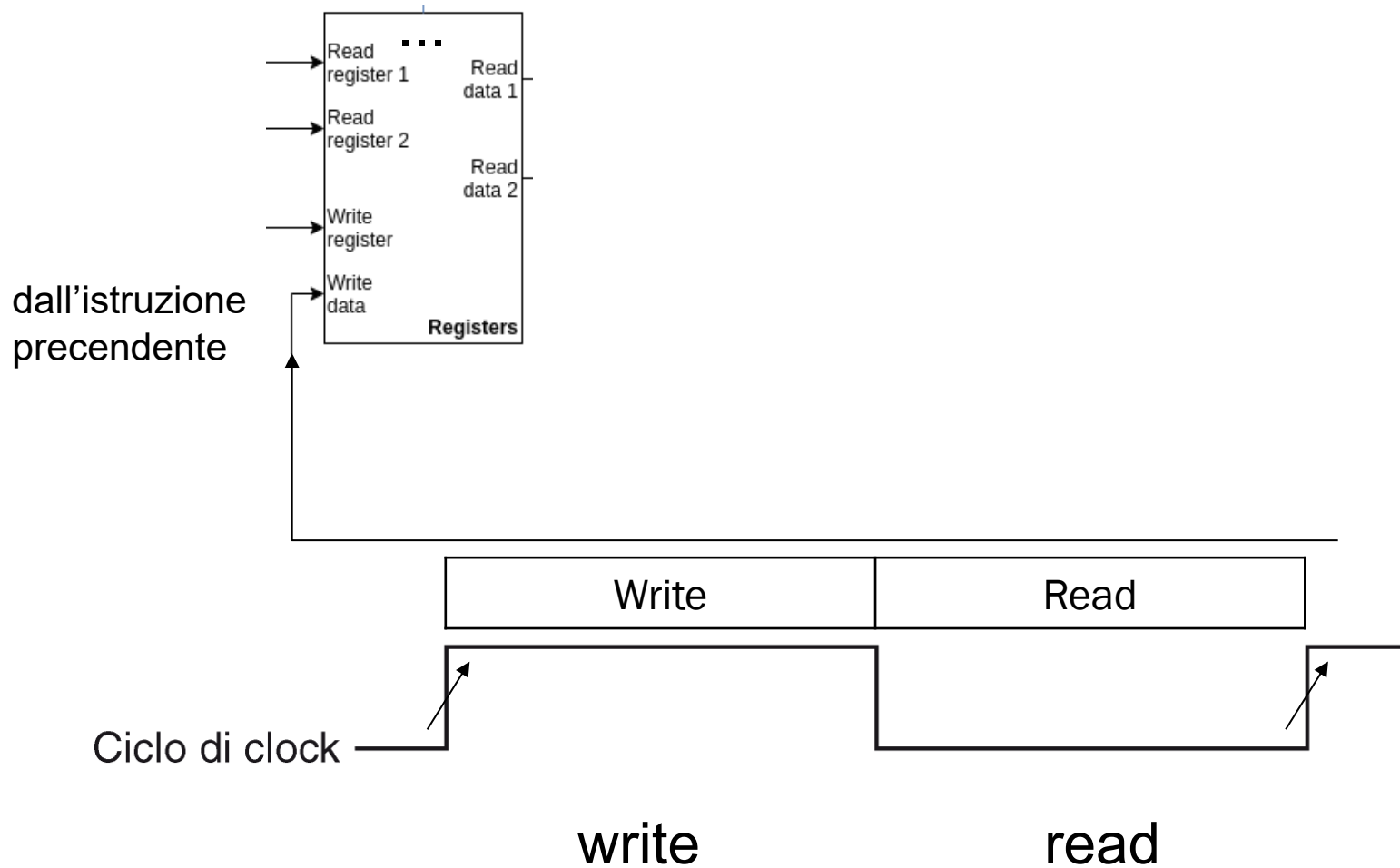


throughput (#istruzioni/fase): 1 con fase da 800 ps, $\frac{1}{4}=0.25$ con fase da 200 ps
 velocità (#istruzioni/[t]): 1/800



throughput (#istruzioni/fase): $\frac{3}{7} \sim 0.43$, velocità (#istruzioni/[t]): 1/466

Chiarimento su Reg e periodo di clock



Ripasso: architetture CISC e RISC

Architettura CISC

(Complex Instruction Set Computer)

- Istruzioni di **dimensione variabile**

Per il fetch della successiva è **necessaria la decodifica**

- **Formato variabile**

Decodifica complessa

- Operandi in memoria

Molti accessi alla memoria per istruzione

- **Pochi registri interni**

Maggior numero di accessi in memoria

- **Modi di indirizzamento complessi**

Maggior numero di accessi in memoria

Durata variabile della istruzione

Conflitti tra istruzioni più complicati

- Istruz. Complesse: pipeline più complicata

Architettura RISC

(Reduced Instruction Set Computer)

- Istruzioni di **dimensione fissa**

Fetch della successiva senza decodifica della prec.

- Istruzioni di **formato uniforme**

Per semplificare la fase di decodifica

- **Operazioni ALU solo tra registri**

Senza accesso a memoria (ALU indipendente da REG)

- **Molti registri interni**

Per i risultati parziali senza accessi alla memoria

- **Modi di indirizzamento semplici**

Con spiazzamento, 1 solo accesso a memoria

Durata fissa della istruzione

Conflitti semplici

- Istruz. semplici: pipeline più veloce

Progettazione del set di istruzioni per pipeline

Vantaggi del RISC

1. Registri sorgente nella medesima posizione
 - Lettura di rs già durante la fase ID
2. Operandi in memoria solo per lw e sw (mai per op. logico-aritmetiche)
 - EXE per calcolare l'indirizzo, MEM per leggere/scrivere (ALU non più impegnata)
3. Un solo risultato, sempre all'ultimo stadio
 - Trasferimento del risultato sempre in coda
4. Allineamento degli operandi in memoria
 - Un singolo accesso in memoria necessario per dato (32 bit)
5. Medesima lunghezza di ogni istruzione
 - v. slide successiva

Circa le dimensioni fisse e variabili

- La fase IF può sovrapporsi alla ID perché l'istruzione ha **dimensione uniforme**
 - Quindi possiamo calcolare il prossimo PC (+4) senza sapere di che tipo di istruzione si tratti
- Nelle architetture con **dimensione delle istruzioni variabile** è necessario aspettare anche la fase ID
 - Quindi la velocità tende ad essere più bassa

CISC									
IF	ID	altre	fasi	...					
		IF	ID	altre	fasi	...			
				IF	ID	altre	fasi	...	

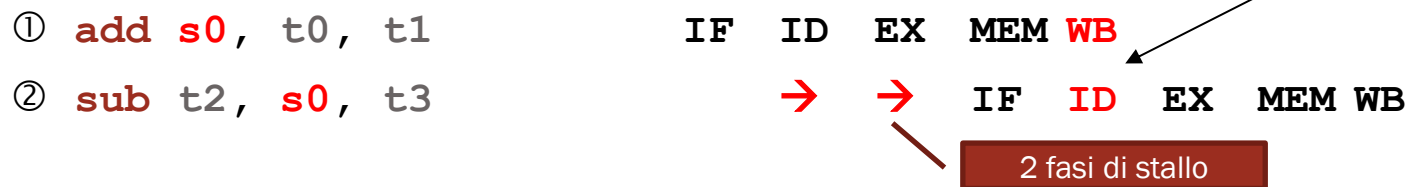
RISC									
IF	ID	EX	MEM	WB					
	IF	ID	EX	MEM	WB				
		IF	ID	EX	MEM	WB			

Criticità (hazard)

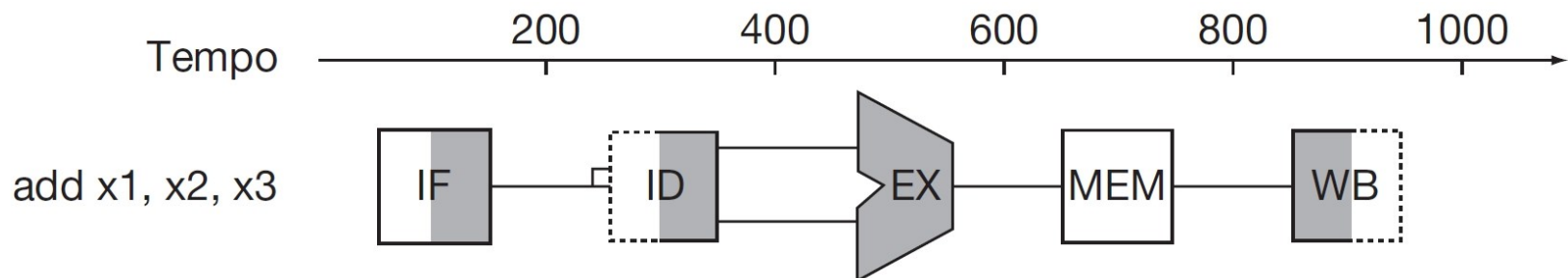
Criticità nella esecuzione (Hazard)

- **strutturali (structural hazard)**: le risorse hardware non sono sufficienti
 - per es., memoria dati e memoria istruzioni in una sola unità (IF/MEM collision) *risolto in fase di design*
- **sui dati (data hazard)**: il dato necessario non è ancora pronto
- **sul controllo (control hazard)**: la presenza di un salto cambia il flusso di esecuzione delle istruzioni

Esempio (data hazard sul registro s0):

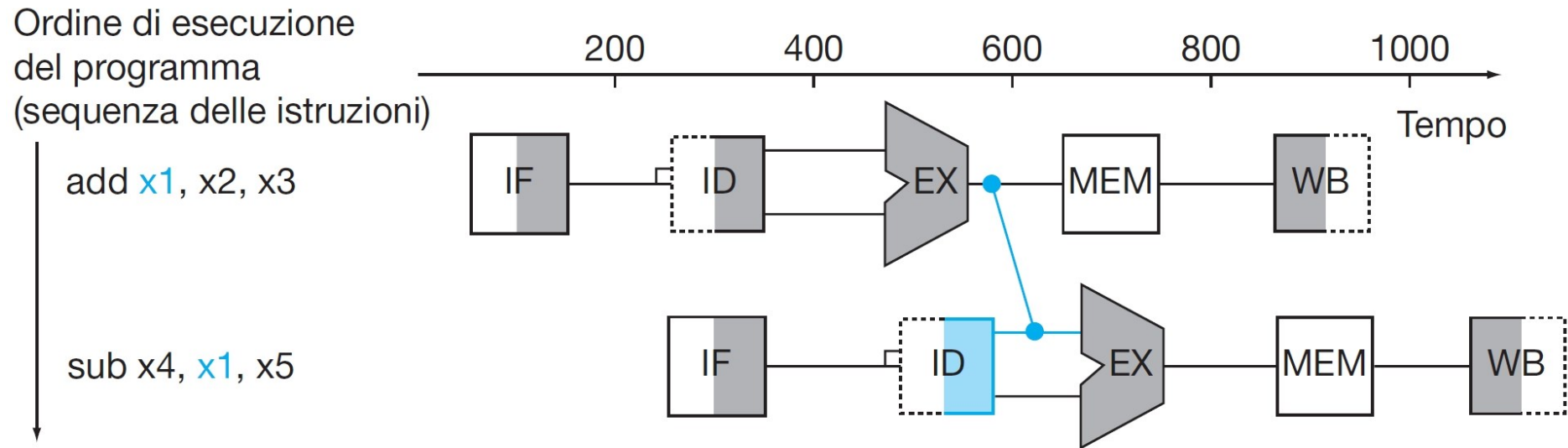


- La ② non può eseguire la lettura degli argomenti se la ① non esegue il WB
 - Le fasi WB della e ID della possono essere sovrapposte se
la WB avviene nella prima metà e ID nella seconda metà del periodo di clock



Propagazione (Bypassing/forwarding)

- In alcuni casi l'informazione necessaria è già presente nella pipeline prima del WB



- Possiamo **inserire nel datapath** delle «**scorciatoie**»
 - recapitano il dato all'unità funzionale che ne ha bisogno senza aspettare la fase di WB
- In questo esempio **NON** è necessario aspettare 2 colpi di clock come nel precedente
 - Possibile quando lo stadio che deve ricevere il dato è **successivo** a quello che lo produce nel **diagramma temporale** della pipeline

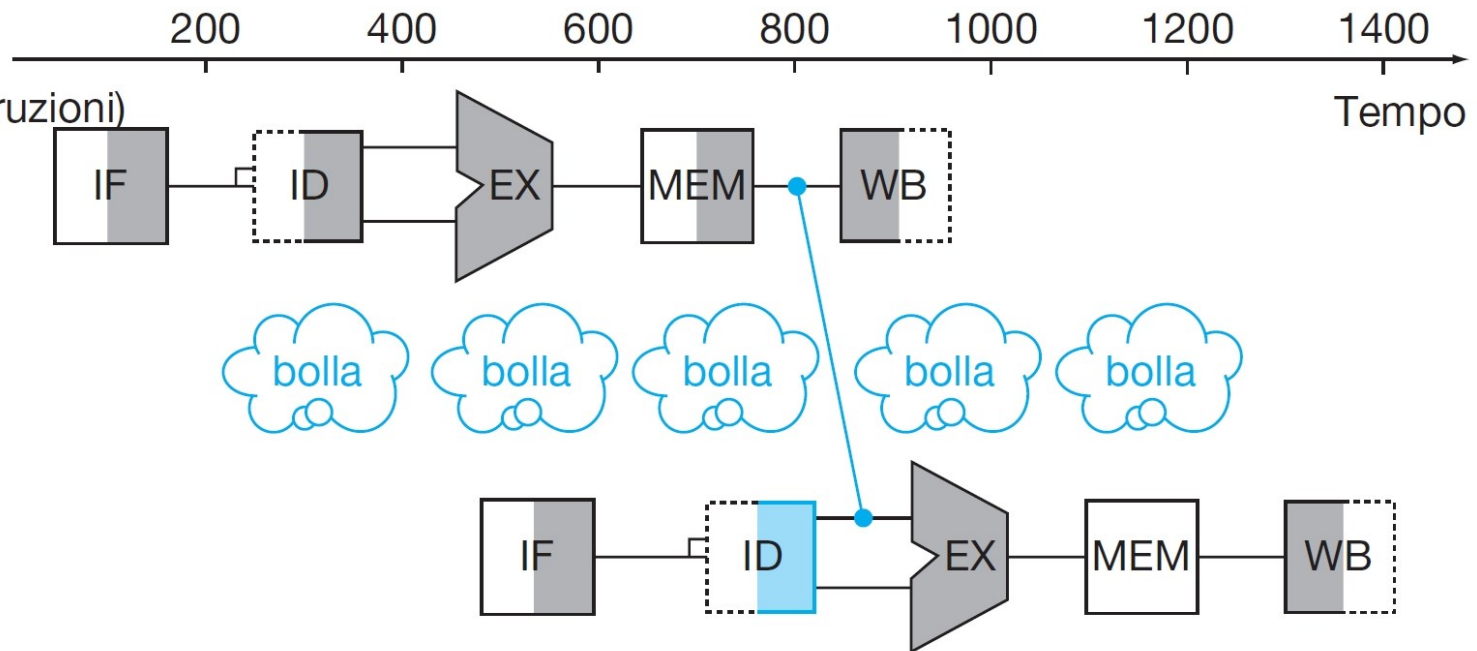
Bolla (bubble)

Se la fase che ha bisogno del dato si trova prima (nel tempo) di quella che lo produce sarà necessario inserire una attesa (stallo o bolla)

Ordine di
esecuzione
del programma
(sequenza delle istruzioni)

lw x1, 0(x2)

sub x4, x1, x5



Riordinamento delle istruzioni

Esempio: $a = b + e$

$c = b + f$

① **lw** t1, 0(t0) IF ID EX MM WB

② **lw** t2, 4(t0) IF ID EX **MM** WB

③ **add** t3, t1, t2 -> IF ID **EX** MM WB

④ **sw** t3, 12(t0) IF ID EX MM WB

⑤ **lw** t4, 8(t0) IF ID EX **MM** WB

⑥ **add** t5, t1, t4 -> IF ID **EX** MM WB

⑦ **sw** t5, 16(t0) IF ID EX MM WB

stallo

stallo

Riordinamento delle istruzioni (con bypass)

Esempio: $a = b + e$
 $c = b + f$

Possibile purché si mantenga la stessa semantica



①	lw	t1,	0 (t0)	IF	ID	EX	MM	WB	
②	lw	t2 ,	4 (t0)		IF	ID	EX	MM	WB
③	add	t3,	t1, t2	->	IF	ID	EX	MM	WB
④	sw	t3,	12 (t0)			IF	ID	EX	MM
⑤	lw	t4 ,	8 (t0)				IF	ID	EX
⑥	add	t5,	t1, t4	->				IF	ID
⑦	sw	t5,	16 (t0)						IF

stallo

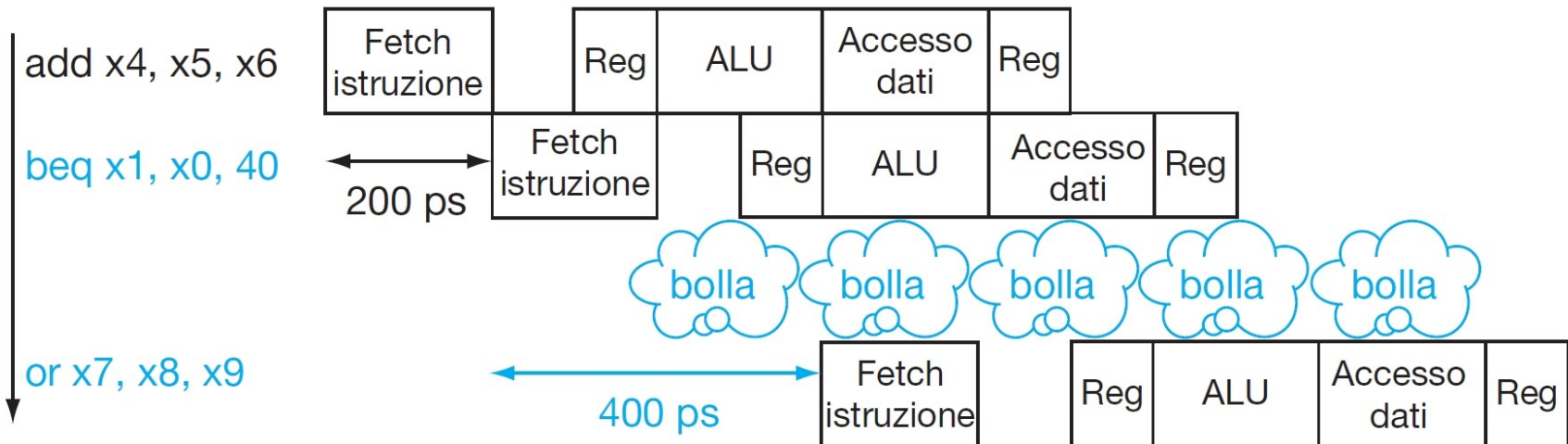
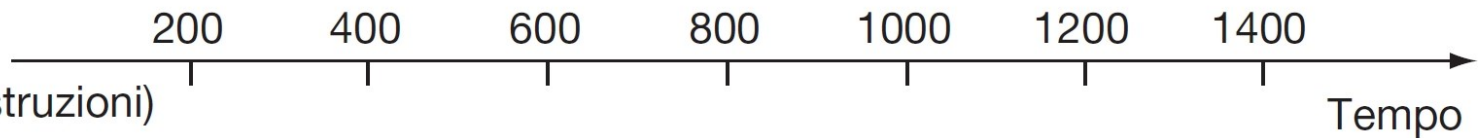
stallo

①	lw	t1,	0 (t0)	IF	ID	EX	MM	WB	
②	lw	t2,	4 (t0)		IF	ID	EX	MM	
⑤	lw	t4,	8 (t0)			IF	ID	EX	
③	add	t3,	t1, t2				IF	ID	
④	sw	t3,	12 (t0)					IF	
⑥	add	t5,	t1, t4						IF
⑦	sw	t5,	16 (t0)						

Hazard sul controllo

- Esempio: salto condizionato (se $x1 == x0$, esegui una `or`; altrimenti vai a 40)
 - L'istruzione seguente (già in pipeline) può dover essere scartata
 - per eseguire quella di destinazione del salto
 - L'istruzione seguente può essere caricata solo dopo che il salto è stato deciso
 - necessari 2 stalli se `beq` viene decisa in EXE o 1 se viene decisa in ID

Ordine di
esecuzione
del programma
(sequenza delle istruzioni)



in questo esempio salto deciso in fase ID

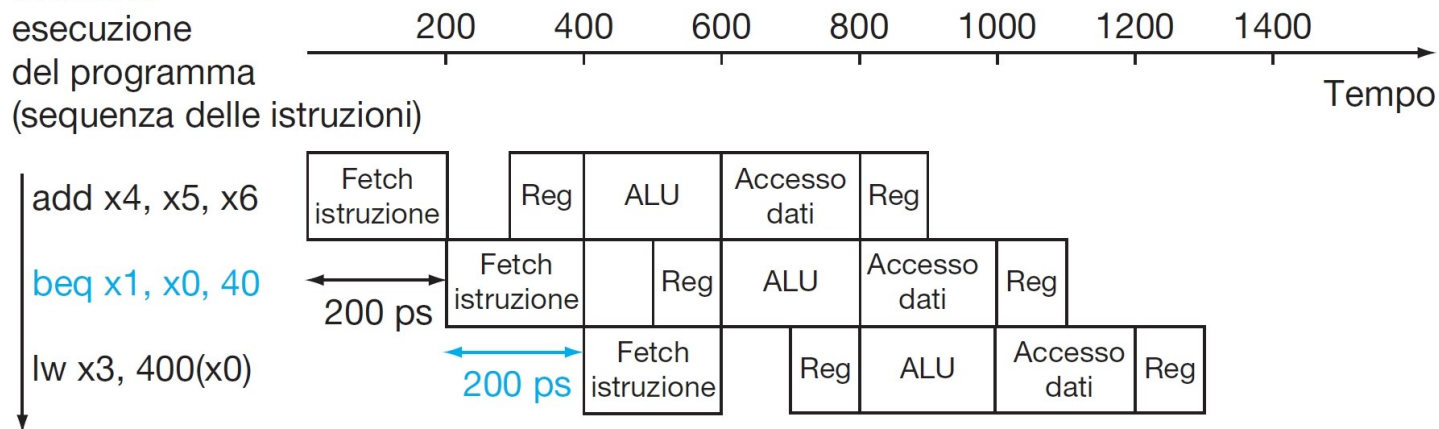
Come mitigare i control hazard

- **Anticipare la decisione (soluz HW):**
 - se beq viene calcolata dalla ALU nella fase EXE ci saranno 2 istruzioni da «buttare»
 - se invece beq viene decisa nella fase ID è sufficiente «buttare» 1 istruzione
- **Ritardare il salto:**
 - se l'istruzione che segue la beq viene SEMPRE eseguita anche se il salto viene fatto, si elimina di fatto lo stallo eseguendo quell'istruz al posto della bolla (non sempre possibile)
- **Branch prediction:** (previsione del salto)
 - la CPU osserva i salti eseguiti e cerca di pre-caricare l'istruzione (seguente o di destinazione) eseguita più spesso
 - static predictors v. dynamic predictors

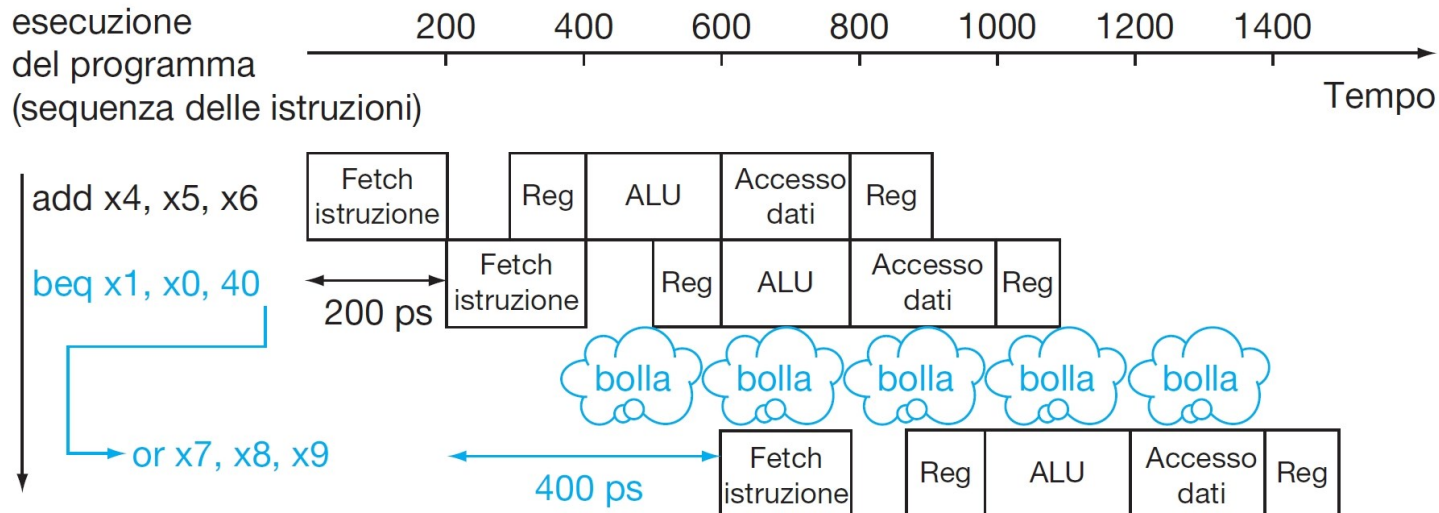
L'implementazione della beq che studiamo assume che il salto non venga eseguito → carica sempre l'istruzione seguente e poi se necessario butta l'istruzione

Effetti della predizione

Ordine di
esecuzione
del programma
(sequenza delle istruzioni)



Ordine di
esecuzione
del programma
(sequenza delle istruzioni)



Esercizio

Per ognuna delle sequenze di istruzioni indicate di seguito, stabilire quale delle tre condizioni seguenti è valida:

1. occorre mettere in stallo la pipeline
2. si possono evitare stalli utilizzando la propagazione (MEM -> EXE o EXE -> EXE)
3. si può eseguire il codice senza stalli né propagazione

Sequenza 1	Sequenza 2	Sequenza 3
<pre>lw t0, 0(t0) add t1, t0, t0</pre>	<pre>add t1, t0, t0 addi t2, t0, 5 addi t4, t1, 5</pre>	<pre>addi t1, t0, 1 addi t2, t0, 2 addi t3, t0, 2 addi t3, t0, 4 addi t5, t0, 5</pre>



Esercizio per casa

Calcolare il numero di cicli necessari ad eseguire le istruzioni seguenti:

- individuare i **data e control hazard**
- per **determinare se con il forwarding possono essere risolti**, tracciare il diagramma temporale della pipeline
- determinare **quali non possono essere risolti e necessitano di stalli** (e quanti stalli)
- tenere conto del tempo necessario a **caricare la pipeline**

Assumere

- che la decision del salto sia presa in fase EXE (e quindi al prossimo ciclo di clock si potrà fare l'IF dell'istruzione corretta)
- che il salto beq non sia ritardato.

Somma un vettore di word

sommaVettore:

```
li t0, 0 # accumul.
li t1, 0x10008028 # fine(10)
li t2, 0x10008000 # ind.base
                    vettore

ciclo: beq t2, t1, fine
        lw t3, (t2)
        add t0, t0, t3
        addi t2, t2, 4
        beq x0,x0,ciclo # "jump"

fine: li a7, 10
      ecall
```